Phase 7 user flow: controlled modernization

Phase 7C now offers evidence-grounded source discovery, generated target proposals, an independent approval gate, and a synthetic migration test in an ephemeral in-memory SQLite database. It does not read live databases or execute production migrations. Phase 7B still provides a boundary preview; actual service extraction remains unimplemented. Phase 7A completion gates and Docker-backed dependency scanning remain as recorded in verification.

Discover and test a Phase 7C data modernization proposal

1. Prepare a source task against an approved architecture version. Include the repository's migration/schema files and relevant Java persistence or SQL-access source in the authorized task snapshot. A full target schema and prewritten ownership rules are not prerequisites.
2. Open **Data migration** and select **Discover source data architecture**. ArchIntel scans supported SQL DDL, Liquibase XML, Java JPA entities, Spring Data repository methods, and visible SQL query patterns. It shows entities, columns, keys, relationships, read/write paths, evidence paths and hashes, parser coverage, ownership hypotheses, confidence, gaps, and the smallest additional input it needs. Runtime database observations are context only and do not prove table identity.
3. Select **Generate target schema and migration proposal**. Choose **Logical schema proposal** to review only, or **Local synthetic SQLite test** if you intend to run the isolated synthetic test. ArchIntel proposes target tables, service assignments where evidence supports them, source-to-target column mappings, type conversions, migration steps, alternatives, risks and unresolved questions. If no schema is discovered, it still returns a proposed empty target and a focused discovery question.
4. Inspect evidence, then revise mappings where required. A mapping revision increments its mapping version and invalidates any prior review. Add only explicitly defined supported invariants (`ROW_COUNT_EQUAL`, `UNIQUE`, `NOT_NULL`, `FOREIGN_KEY_VALID`); unknown tables/columns are rejected.
5. A different human with `SECURITY_ADMIN` and source access reviews the exact current proposal. They must record an ownership decision for every discovered resource, including `SHARED` when ownership remains shared, and acknowledge every remaining non-hard risk.

The decision is stored with its rationale and bound to the resulting plan fingerprint. Approval authorizes **synthetic test execution only**.

6. Enter synthetic fixture rows in the test form and select **Run synthetic migration test**. ArchIntel creates source and target tables in an in-memory SQLite database, converts and copies the fixture, then checks counts, key constraints, referential integrity, supported conversions and defined invariants. The database is discarded when the process closes. Raw fixture rows and database connection strings are not persisted. Results are limited to the submitted synthetic fixture and checks shown.

Proposals are currently generated by deterministic evidence rules; ArchIntel does not claim to infer business meaning or use a generative model in this workflow. Production execution is disabled. There is no live database adapter, production credential flow, cutover action, or backup/restore guarantee. A local `PASS` does not establish compatibility with Concord, PostgreSQL or another database engine, nor prove full application behavior. A production path requires separately authorized database-specific validation, representative-data authorization, recovery drills and a distinct production approval system.

If discovery is incomplete, include the smallest missing schema/migration/access-path source identified by the report and rerun discovery. Do not manually invent facts to make the proposal pass.

Preview an extraction boundary

1. Open a prepared source task and choose **Extraction plan**.
2. Enter a review note and select **Preview extraction boundary**. The preview can run before prerequisites are complete so it can report missing evidence.
3. Review the candidate target classes/packages and source paths, incoming/outgoing dependencies, cross-boundary cycles, and blockers.
4. Treat unresolved classes, incomplete source snapshots, unreviewed contracts, or missing data-ownership review as blockers. The preview is bound to the selected architecture version and task snapshot and becomes stale if those inputs change.

This preview does not edit source or run a build. Candidate source paths use package/type filename suffixes and are not verified by a Java parser. Do not treat a clear preview as extraction approval; extraction generation and validation are still pending.

Generate a controlled Java refactor

1. Include the implementation Java file and the new interface path in the task's allowed files. The interface path must not already exist.
2. Open **Patch review** and select the implementation and interface paths.

3. Enter a Java interface name and one or more unique public instance method names. Overloaded, missing, static or non-public methods are rejected.
4. Select **Generate interface changeset**. ArchIntel runs JavaParser 3.28.2 inside the pinned offline container. It parses one top-level implementation class, creates the interface signatures and adds `implements <Interface>` to the class.
5. Inspect the exact patch and use the existing review/stage/build flow. Generation never stages, merges or deploys code automatically.

The interface filename must match its public type name and sit beside the implementation. Isolated execution must be enabled in workspace policy. Imported signature types are copied. Generic implementation classes, nested type declarations, static imports, name conflicts and overloaded selections are rejected. The recipe does not rewrite consumers or claim architecture improvement. A human chooses the methods and reviews the output; compile, contract, architecture and security checks remain required.

Scan the candidate for security findings

After a successful captured candidate build, open **Validation**, enter a note and select **Run Java security scan**. ArchIntel uses a separate, immutable SpotBugs/FindSecBugs image with a server-owned security filter. It records rule identifiers, affected classes and priorities. Any finding fails this conservative gate; missing classes or analysis errors are inconclusive. A failed retry replaces the previous result for validation.

If the operator has provisioned the Trivy image and databases, the same action also scans packages inside the captured JAR/WAR. No packages identified is inconclusive. Database metadata older than seven days is rejected. This identifies packaged dependencies; it does not establish a complete inventory of dependencies supplied externally at runtime. Complete dependency coverage remains required for final acceptance.

Run **Compare validation evidence** after the scan. The Java and packaged-dependency gates are separate. A clean report is limited to the scanner's checks and captured artifact; it does not prove the application is secure.

Compare a staged change

1. Open **Source tasks**. Prepare a task using the Phase 6 guide and an approved modernization plan.
2. Define and review behavioral contracts that reference captured test files and exact JUnit case names.
3. Run the baseline build. It must reproduce the analyzed artifact hash; use a reproducible build and the appropriate offline dependency cache.
4. Propose a patch, inspect it, approve staging and stage it in the isolated workspace.

5. Run the candidate build using the same pinned container image.
6. Open **Validation**, enter an operation note and select **Start or refresh architecture re-analysis**. This queues the captured candidate JAR/WAR for the existing static scanner. Older candidate builds without artifact capture must be rebuilt first.
7. Allow the scan to finish, then select the same button to finalize the comparison. Inspect added/removed classes, dependencies, cycles and findings. Different scanner versions, algorithms or analysis settings make the comparison inconclusive. A failed scan requires investigation and a new source task; this checkpoint does not automatically retry scans.
8. Select **Compare validation evidence** and inspect every gate and contract. `PASS` applies only to that check. `NOT_RUN` means required evidence is missing. `INCONCLUSIVE` means a reliable comparison cannot be made. `FAIL` requires investigation.

The comparison uses the latest attempt of each build kind, even when that attempt failed. Changed or deleted contract tests, missing or duplicate exact case names, removed baseline cases, skipped candidate cases, changed source hashes and different build images block passing contract comparison. New candidate cases are allowed, but must pass.

A comparison becomes **STALE** after its inputs or task state change. Rebuild as needed, then compare again. Repeating an unchanged comparison does not add duplicate history.

Architecture re-analysis can pass when a completed, comparable scan binds to the latest candidate build and exact staged patch. This means the comparison was performed, not that the architecture improved. Generated-code security remains `NOT_RUN` and overall status remains **BLOCKED**. Passing tests do not prove complete behavioral equivalence or architecture improvement. There is no final approval, merge or deployment action.

Review improvement and approve a candidate

After validation has fresh PASS results for every required gate, open the Validation tab and inspect the measured expected-improvement metric selected from the approved plan decision. The comparison uses the baseline and candidate static evidence; it preserves the baseline class inventory and reports PASS, FAIL, or INCONCLUSIVE. An authorized reviewer can accept only a measured PASS, bound to its exact fingerprint. If any required evidence is missing, stale, incomplete, or fails, candidate publication stays disabled.

Select **Publish validated candidate version** only after the required gates and impact review pass. This creates a canonical architecture version linked to its parent and the candidate analysis. Then a different authorized security administrator reviews the current evidence and selects **Record final human approval**. Approval is bound to the current validation and gate fingerprints; any change to the candidate or evidence makes the result stale and requires revalidation. These actions approve the reviewed transformation record; ArchIntel does not merge code or deploy it.

The candidate is stored separately from uploaded application artifacts and does not become the default input for the application's ordinary Analyze action. Its analysis ID is displayed for provenance. Candidate JAR/WAR captures are retained in local task storage, up to 32 MiB per build and the existing 50-build task limit; this checkpoint has no automatic artifact-retention purge.

Recovery

Existing pause, resume and private-workspace rollback remain available. Rollback restores captured task files; it does not overwrite the original checkout and does not undo database changes. A rolled-back task retains its historical comparison marked stale.